DAY 04 OF 10

Lists & Tuples

Store many values together and process them

Today's agenda

1. Creating and indexing lists
2. List methods
3. Looping over lists
4. List comprehension
5. Tuples and unpacking

Python in 10 Days
Lakshmi Prasad Gondinigari

Lists

- An ordered, changeable collection written in [ ]
- Can hold mixed types, even other lists
- Index and slice like strings
- Mutable: you can change items in place
- len() gives the number of items

Python

```python
fruits = ["apple", "banana", "mango"]
print(fruits[0])
print(fruits[-1])
print(fruits[0:2])
fruits[1] = "kiwi"
print(fruits)
print(len(fruits))
```

OUTPUT

```
apple
mango
['apple', 'banana']
['apple', 'kiwi', 'mango']
3
```

List methods

DAY 04

Method	What it does	Example
append(x)	add at the end	a.append(7)
insert(i, x)	add at position i	a.insert(0, 1)
extend(list)	add many items	a.extend([3, 3])
remove(x)	delete first x	a.remove(3)
pop(i)	delete and return item (last by default)	a.pop()
sort() / reverse()	order in place	a.sort()
index(x) / count(x)	position / how many	a.count(3)

> sorted(a) returns a new sorted list. a.sort() changes the list itself and returns None.

Methods in action

- Follow the list after every step
- sum(), min(), max() work on number lists
- Copy a list with a.copy() or a[:]. Plain b = a shares the same list!

Python

```python
nums = [5, 2, 9]
nums.append(7)        # [5, 2, 9, 7]
nums.insert(0, 1)     # [1, 5, 2, 9, 7]
nums.extend([3, 3])   # [1, 5, 2, 9, 7, 3, 3]
nums.remove(3)        # [1, 5, 2, 9, 7, 3]
last = nums.pop()     # last = 3
nums.sort()           # [1, 2, 5, 7, 9]
nums.reverse()        # [9, 7, 5, 2, 1]
print(nums, sum(nums), max(nums))
```

OUTPUT

```
[9, 7, 5, 2, 1] 24 9
```

Looping over lists

DAY 04

- for item in list: gives each value
- enumerate() gives index and value
- zip() walks two lists together
- in checks membership

Python

```python
names = ["Asha", "Ravi", "Sam"]
marks = [82, 67, 91]
for i, name in enumerate(names, start=1):
    print(i, name)
for n, m in zip(names, marks):
    print(n, m)
print("Ravi" in names)
```

OUTPUT

```
1 Asha
2 Ravi
3 Sam
Asha 82
Ravi 67
Sam 91
True
```

List comprehension

- A short way to build a new list from a loop
- Pattern: [expression for item in sequence if condition]
- Faster to write and usually easier to read for simple cases
- If it gets complicated, use a normal loop

Python

```python
squares = [n * n for n in range(1, 6)]
evens = [n for n in range(10) if n % 2 == 0]
names = ["asha", "ravi"]
caps = [x.title() for x in names]
print(squares)
print(evens)
print(caps)
```

OUTPUT

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
['Asha', 'Ravi']
```

Nested lists

- A list of lists models a table or a matrix
- Use two indexes: matrix[row][column]
- Loop with a nested for

Python

```python
matrix = [[1, 2, 3],
          [4, 5, 6]]
print(matrix[1][2])
for row in matrix:
    for value in row:
        print(value, end=" ")
    print()
```

OUTPUT

```
6
1 2 3
4 5 6
```

Tuples

- Like a list but immutable: cannot be changed after creation
- Written in ( ). A single-item tuple needs a comma: (5,)
- Faster and safer for fixed data (coordinates, RGB colours)
- Unpacking assigns each item to a variable
- Functions often return tuples

Python

```python
point = (10, 20)
x, y = point
print(x, y)
# point[0] = 5   -> TypeError

name, age, city = ("Asha", 21, "Pune")
print(city)
a, b = 1, 2
a, b = b, a        # swap
print(a, b)
```

OUTPUT

```
10 20
Pune
2 1
```

Practice time

DAY 04

1. Find the largest and smallest number in a list without using max/min
2. Remove duplicates from a list while keeping the order
3. Given marks of 5 students, print average and the students above average
4. Create a list of squares of odd numbers up to 20 using comprehension
5. Rotate a list by one position (last item goes first)

Challenge: merge two sorted lists into one sorted list

Day 4 recap

DAY 04

- Lists are ordered and mutable; tuples are ordered and immutable
- Key methods: append, insert, extend, remove, pop, sort, reverse
- enumerate and zip make loops cleaner
- List comprehension builds lists in one line

Homework: finish practice. Next: Dictionaries and Sets